STORED PROCEDURE



Stored Procedure

- **Collection of SQL statements**
- **Can accept parameters**
- **Contain programming statements that perform operations in the database, including calling other procedures.**



Creating a Stored Procedure

```
CREATE PROC [ EDURE ] procedure_name
   [ { @parameter data_type }]


AS


   sql_statement
```

Creating a Stored Procedure

AS

- **Specifies the actions the procedure is to take**

Sql_statement

- **Is any number and type of Transact-SQL statements to be included in the procedure**



Simple Stored Procedure

```
CREATE PROCEDURE SP_LIST_PRODUCT_INFO
AS
SELECT c.categoryname, p.productname,
            p.unitprice
FROM production.category c
INNER JOIN production.products p
   ON c.categoryid = p.categoryid
ORDER BY c.categoryname
GO
```

Stored Procedure Execution

```
EXECUTE SP_LIST_PRODUCT_INFO
```

```
EXEC SP_LIST_PRODUCT_INFO
```

```
SP_LIST_PRODUCT_INFO
```



Stored Procedure with Parameters

```
CREATE PROCEDURE SP_LIST_PRODUCT_INFO
        @p_categoryID numeric
AS
SELECT c.categoryname, p.productname,
            p.unitprice
FROM production.category c
INNER JOIN production.products p
   ON c.categoryid = p.categoryid
WHERE p.categoryid = @p_categoryID
ORDER BY p.productname
GO
```

Stored Procedure Execution

```
EXECUTE SP_LIST_PRODUCT_INFO 7
```

```
EXECUTE SP_LIST_PRODUCT_INFO  @p_categoryID = 7
```



```
CREATE PROCEDURE SP_LIST_PRODUCT_INFO
                @p_categoryID numeric = NULL
AS
IF @p_categoryID IS NULL
      SELECT c.categoryname, p.productname,
                        p.unitprice
      FROM production.category c
      INNER JOIN production.products p
        ON c.categoryid = p.categoryid
      ORDER BY p.productname
ELSE
      SELECT c.categoryname, p.productname,
                        p.unitprice
      FROM production.category c
      INNER JOIN production.products p
        ON c.categoryid = p.categoryid
      WHERE p.categoryid = @p_categoryID
      ORDER BY p.productname
GO
```

Altering Stored Procedurec

- **Alters a previously created procedure, created by executing the CREATE PROCEDURE statement, without changing permissions and without affecting any dependent stored procedures**

Altering Stored Procedure

```
ALTER PROC [ EDURE ] procedure_name
   [ { @parameter data_type }]

AS

   sql_statement
```

Dynamic SQL

- Dynamic SQL is T-SQL code assembled into a character string, interpreted as a command and executed.
- Dynamic SQL provides flexibility for administrative, programming tasks.
- Two methods for dynamically executing SQL statements:
  - EXEC command can accept a string as input in parentheses. No parameters may be passed in.
  - System stored procedure sp_executesql (preferred) supports parameters
- Beware of risks from unvalidated inputs in dynamic SQL!

Writing Queries with Dynamic SQL

- Using sp_executesql
  - Accepts string as code to be run
  - Allows parameterized code without risk of SQL injection
  - Can perform better than EXEC due to query plan reuse

```
DECLARE @sqlcode AS NVARCHAR(256)
SET @sqlcode =       N'<code_to_run>';
EXEC sp_executesql  @sqlcode;
```

```
DECLARE @sqlcode AS NVARCHAR(256)
SET @sqlcode  = N'SELECT GETDATE() AS dt';
EXEC sp_executesql  @sqlcode;
```

```
CREATE PROCEDURE SP_LIST_PRODUCT
 @p_categoryID numeric = NULL
AS
     DECLARE @sql_code as NVARCHAR(500)
     SET  @sql_code   = N'SELECT c.categoryname, p.productname,    p.unitprice
          FROM production.category c
          INNER JOIN production.products p
            ON c.categoryid = p.categoryid
          ORDER BY p.productname'
     IF @p_categoryID IS NULL
     BEGIN
              SET @sql_code   =  @sql_code +
                    N' WHERE p.categoryid =  ' + CAST(@p_categoryID  AS NVARCHAR)
     END
    SET  @sql_code = @sql_code +  N' ORDER BY p.productname'
    EXEC SP_EXECUTESQL @sql_code
GO
```